

Connecting a buzzer directly to a Raspberry Pi 5 GPIO pin causes continuous beeping because the Pi's pins operate at 3.3V and can only safely supply about 16mA of current. Direct connections either pull the pin into a floating state or draw too much current, which risks permanently burning out the Raspberry Pi's processor.

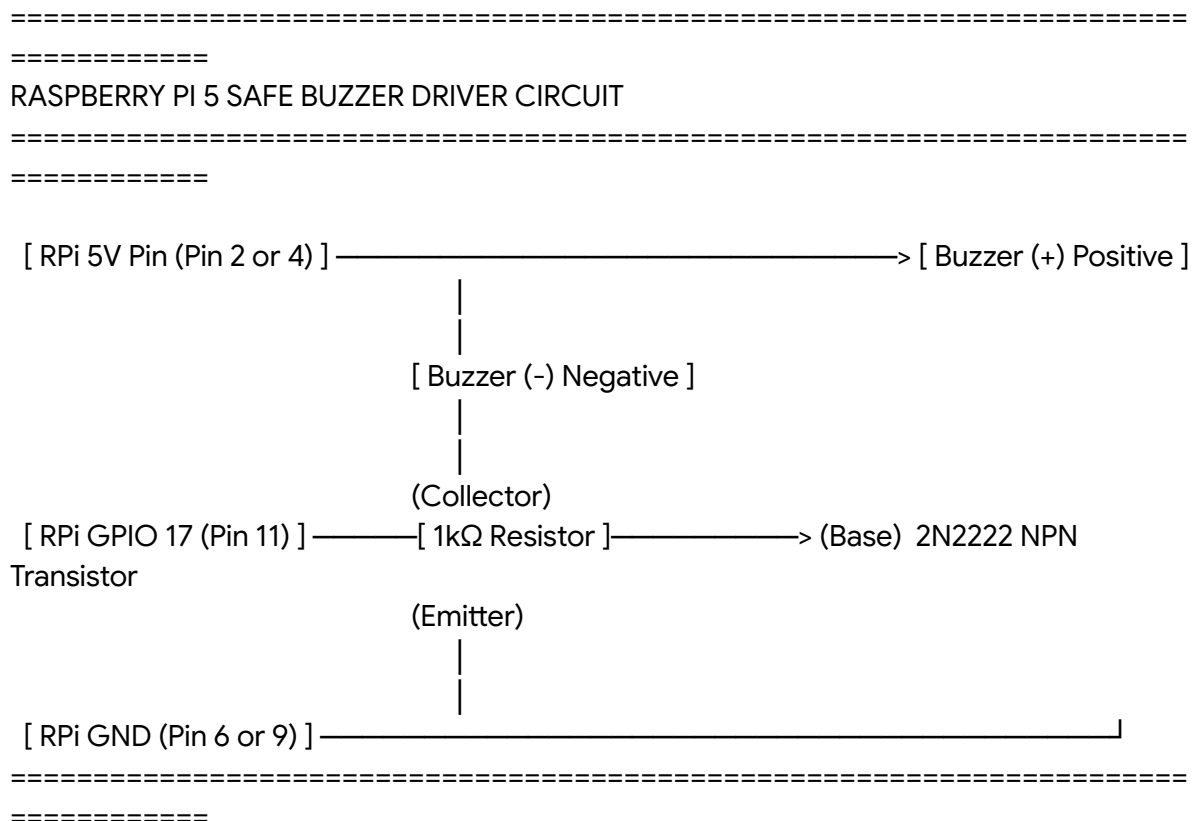
To safely switch a 5V buzzer using a 3.3V Raspberry Pi signal, you must use an **NPN Transistor** acting as a digital switch, paired with a **Current-Limiting Resistor**.

Required Components

- **1x NPN Transistor (2N2222 or BC547):** Acts as the heavy-duty switch to route 5V power to the buzzer without touching the Pi's delicate logic pins.
- **1x $1\text{k}\Omega$ Resistor:** Protects the Raspberry Pi GPIO pin by limiting the current flowing into the transistor base.
- **1x 5V Active Buzzer:** Generates sound automatically when 5V DC is applied.
- *(Optional)* **1x 1N4007 Diode:** Wired across the buzzer to absorb reverse-voltage spikes (highly recommended if the buzzer is magnetic rather than piezoelectric).

Raspberry Pi 5 Buzzer Wiring Schematic

Plaintext



How it Works:

When the Raspberry Pi sends a HIGH (3.3V) signal from GPIO 17, it passes safely through the Ω resistor and turns the transistor "ON". The transistor then completes the circuit to Ground, allowing the 5V power from the Pi to flow through the buzzer and make it scream.

Python Control Script (gpiozero)

Since you are using a Raspberry Pi 5, the modern standard for GPIO control is the gpiozero library. This script handles both the single door beep and the continuous emergency siren.

```
Python
from gpiozero import Buzzer
from time import sleep

# Initialize the buzzer on GPIO 17
alarm_buzzer = Buzzer(17)

def door_beep():
    """Single 100ms beep for door open/close events."""
    print("[EVENT] Main Door Toggled")
    alarm_buzzer.on()
    sleep(0.1)
    alarm_buzzer.off()

def emergency_alarm():
    """Continuous pulsing alarm for Fire/Earthquake."""
    print("💣 [ALARM] EMERGENCY DETECTED! 💣")
    try:
        while True:
            alarm_buzzer.on()
            sleep(0.5)
            alarm_buzzer.off()
            sleep(0.5)
    except KeyboardInterrupt:
        alarm_buzzer.off()
        print("Alarm silenced.")

# --- Test the functions ---
if __name__ == '__main__':
    # Test door beep
    door_beep()
```

```
sleep(2)
```

```
# Trigger emergency alarm  
emergency_alarm()
```

Raspberry Pi MQTT Buzzer Node

Master Cheat Sheet — Code, Configuration, Commands, and Home Assistant Automations

1. Python Script (The Engine)

File Location: /home/redwannabil/ha_buzzer_node.py

Purpose: Listens to Home Assistant and translates MQTT messages into physical buzzer sounds.

```
import time
import paho.mqtt.client as mqtt
from gpiozero import Buzzer

# =====
# CONFIGURATION
# =====
MQTT_SERVER = "192.168.0.40"
MQTT_PORT = 1883
MQTT_USER = "redwanmqtt"    # Your verified HA Mosquitto user
MQTT_PASS = "abcd2005-"     # Your HA Mosquitto password

COMMAND_TOPIC = "home/redwannabil/buzzer/cmd"
STATE_TOPIC = "home/redwannabil/buzzer/state"

# Initialize Buzzer on GPIO 17
buzzer = Buzzer(17)

# =====
# MQTT CALLBACKS
# =====
def on_connect(client, userdata, flags, reason_code, properties):
    print(f"[MQTT] Connected with result code {reason_code}")
    if reason_code == 0:
        client.subscribe(COMMAND_TOPIC)
        client.publish(STATE_TOPIC, "OFF", retain=True)
    else:
        print("Connection failed! Please check MQTT_USER and MQTT_PASS.")

def on_message(client, userdata, msg):
    payload = msg.payload.decode("utf-8")
    print(f"[CMD IN] {payload}")

    if payload == "ON":
        buzzer.on()                # Solid continuous tone
        client.publish(STATE_TOPIC, "ON", retain=True)

    elif payload == "PULSE":
        buzzer.beep(0.5, 0.5)      # Continuous pulsing siren
        client.publish(STATE_TOPIC, "PULSE", retain=True)

    elif payload == "OFF":
        buzzer.off()               # Silence
        client.publish(STATE_TOPIC, "OFF", retain=True)

    elif payload == "BEEP":
        # Non-blocking quick beep for doors (doesn't change overall state)
        buzzer.beep(on_time=0.1, off_time=0.1, n=1, background=True)

# =====
# MAIN LOOP
# =====
client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2, client_id="PiServer_Buzzer_Node")
client.username_pw_set(MQTT_USER, MQTT_PASS)
client.on_connect = on_connect
client.on_message = on_message

print("[SYSTEM] Starting Pi MQTT Buzzer Service...")
client.connect(MQTT_SERVER, MQTT_PORT, 60)
client.loop_forever()
```

2. Linux Background Service (Systemd)

File Location: /etc/systemd/system/pi_buzzer.service

Purpose: Keeps the Python script running 24/7 in the background and restarts it if it crashes.

[Unit]

```
Description=Home Assistant MQTT Buzzer Node
After=network-online.target
```

```
[Service]
# The "-u" forces unbuffered output so you can see live logs!
ExecStart=/usr/bin/python3 -u /home/redwannabil/ha_buzzer_node.py
WorkingDirectory=/home/redwannabil
StandardOutput=inherit
StandardError=inherit
Restart=always
User=redwannabil
```

```
[Install]
WantedBy=multi-user.target
```

3. Essential Linux Management Commands

Run these commands in the Raspberry Pi terminal.

```
# Apply changes to the .service file
sudo systemctl daemon-reload

# Restart the service (apply Python code changes)
sudo systemctl restart pi_buzzer.service

# Start the service automatically on Raspberry Pi boot
sudo systemctl enable pi_buzzer.service

# View live real-time logs (crucial for debugging)
sudo journalctl -u pi_buzzer.service -f
```

4. Home Assistant Configuration (configuration.yaml)

File Location: /config/configuration.yaml (inside Home Assistant)

Purpose: Creates the manual switch.piserver_buzzer entity for the dashboard.

```
mqtt:
  switch:
    - name: "PiServer Buzzer"
      state_topic: "home/redwannabil/buzzer/state"
      command_topic: "home/redwannabil/buzzer/cmd"
      payload_on: "ON"      # Sends solid tone command
      payload_off: "OFF"    # Sends silence command
      state_on: "ON"
      state_off: "OFF"
      icon: mdi:alarm-bell
      unique_id: "piserver_buzzer_node"
```

5. Home Assistant Dashboard Card

Location: Dashboard → Edit → Add Card → Manual YAML

Purpose: A clean manual button to activate the solid emergency tone.

```
type: button
entity: switch.piserver_buzzer
name: PiServer Buzzer (Manual)
icon: mdi:bullhorn
show_state: true
tap_action:
  action: toggle
hold_action:
  action: none
```

6. Home Assistant Automations (Custom Sound Triggers)

Location: Settings → Automations → Create Automation → Edit in YAML

Purpose: Dynamically trigger different sounds such as PULSE or BEEP from smart-home sensors.

Example A: Trigger a short BEEP when a door opens

```
alias: "Door Opened -> Short Beep"
trigger:
  - platform: state
    entity_id: binary_sensor.front_door
    to: "on"
action:
```

```
- action: mqtt.publish
  data:
    topic: home/redwannabil/buzzer/cmd
    payload: "BEEP"
```

Example B: Trigger the PULSE siren if a Gas/Fire Sensor goes off

```
alias: "Fire Detected -> Pulse Siren"
trigger:
  - platform: state
    entity_id: binary_sensor.kitchen_gas_leak
    to: "on"
action:
  - action: mqtt.publish
    data:
      topic: home/redwannabil/buzzer/cmd
      payload: "PULSE"
```

Example C: Silence the Buzzer automatically when the Fire is cleared

```
alias: "Fire Cleared -> Turn Off Siren"
trigger:
  - platform: state
    entity_id: binary_sensor.kitchen_gas_leak
    to: "off"
action:
  - action: mqtt.publish
    data:
      topic: home/redwannabil/buzzer/cmd
      payload: "OFF"
```

MQTT Command	Buzzer Behavior	Reported State
ON	Solid continuous tone	ON
PULSE	Continuous pulsing siren	PULSE
OFF	Silence	OFF
BEEP	One short non-blocking beep	No state change

Note: The document preserves the supplied credentials and configuration exactly as provided. Treat MQTT credentials as sensitive and change them if this PDF is shared publicly.